VOLUME 01 OF 10

Foundations and Your First Game

Beginner / early undergraduate

Learn to reason about programs, run code, debug, test, use Git and complete a tiny game.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 01 Starting as an engineer

01.1 What a program actually does • 01.2 Files, terminals and reproducible execution • 01.3 Trace before running • 01.4 Units, budgets and engineering notebooks

Module 02 Values, decisions and repetition

02.1 Types and expressions • 02.2 Conditions and boundary cases • 02.3 Loops and invariants • 02.4 Functions and contracts

Module 03 Organizing data and small programs

03.1 Sequences, dictionaries and identity • 03.2 Decomposition, modules and interfaces • 03.3 Files, serialization and validation • 03.4 Classes, composition and state machines

Module 04 Debugging, testing and collaboration

04.1 Debugging as an experiment • 04.2 Unit, integration and property tests • 04.3 Git, branches and recoverable changes • 04.4 Review, documentation and responsible AI help

Module 05 A complete first game

05.1 The update loop and command model • 05.2 Collision rules and complete behavior • 05.3 Deterministic replay and simple measurement • 05.4 Finishing, usability and the first portfolio artifact

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 01

Starting as an engineer

Prerequisites None. Basic reading, arithmetic and the ability to save a file.

Four lessons

01.1 What a program actually does

Explain state, instructions, input and output using a tiny game.

01.2 Files, terminals and reproducible execution

Run a saved Python program and distinguish a shell from an interpreter.

01.3 Trace before running

Predict a program by following assignments in order.

01.4 Units, budgets and engineering notebooks

Use units and simple arithmetic to check game calculations.

Module project

Create a learning repository with a working motion program, an environment record, a prediction table and a one-page explanation of the input-to-display path.

Core references

R03 Harvard University — CS50 Introduction to Programming with Python

R04 Python Software Foundation — Python tutorial and language documentation

R15 MIT OpenCourseWare — 6.004 Computation Structures

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 01.1 • MODULE 01

What a program actually does

Learning goal Explain state, instructions, input and output using a tiny game.

A program is a set of instructions that transforms information. Its state is the information that must persist: a player position, a health value, or a list of enemies. An input is a new observation or command. An output might be a picture, sound, file or network message. A game repeatedly reads inputs, changes state and produces outputs. The picture is a representation of the state, not the state itself.

A processor executes machine instructions; an operating system manages resources and access to devices. A game engine is software that organizes recurring tasks such as rendering and asset loading. A graphics processor is hardware that executes suitable workloads. Keep these layers distinct: changing a game rule is not the same activity as designing a processor.

WORKED EXAMPLE

For a player at x = 2, the command move-right by 3 units produces x = 5. Drawing the character twice must not move it twice. Rendering observes state; the simulation changes it.

PRACTICE + PASS CRITERION

On paper, trace five commands: right 3, left 1, right 0, left 4, right 2, starting at x = 2. Draw boxes for input, simulation state and output. Save the trace. Pass: every state transition is explainable without running code.

ASSIGNED STUDY

R03: introductory functions/variables lecture; R15: Basics of Information.

CHECK YOUR UNDERSTANDING

1. What is the difference between state and a displayed image?

2. What final position results from the five lab commands?

3. Why should drawing a character not update its position?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 01.2 • MODULE 01

Files, terminals and reproducible execution

Learning goal Run a saved Python program and distinguish a shell from an interpreter.

A source file stores the instructions you write. A terminal presents a command-line interface; its shell starts programs and manages paths. The Python interpreter is a program that runs Python instructions. A shell prompt is not a Python prompt. Commands such as cd belong in the shell; expressions such as print(2 + 3) belong in Python code.

A working directory tells a process where relative paths begin. Many apparent programming errors are actually wrong-directory or wrong-file errors. Reproducibility starts by recording the exact command, file and tool version. Use a dedicated course folder. Do not run copied administrative commands or upload private files merely to make a tutorial work.

WORKED EXAMPLE

With day01_motion.py in the current folder, python3 day01_motion.py runs that file on many macOS/Linux setups. On Windows, py -3 day01_motion.py is a common launcher form. First confirm the available command with its --version option.

PRACTICE + PASS CRITERION

Follow START_HERE. Run the included program twice from the same folder; save the command and output in your journal. Intentionally misspell the filename, read the error, then repair it. Pass: you can explain which program produced the error.

ASSIGNED STUDY

R04: Using the Python Interpreter; installation links in START_HERE.

CHECK YOUR UNDERSTANDING

1. What does a relative filename depend on?

2. Is cd a Python statement in an ordinary script?

3. What three facts help another person reproduce a run?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 01.3 • MODULE 01

Trace before running

Learning goal Predict a program by following assignments in order.

An assignment evaluates the expression on the right and binds the result to the name on the left. In x = x + 1, the right-hand x means the old value. This is not an algebraic equation claiming that x equals x plus one forever. The order of instructions matters because later instructions can read changed state.

A trace table records values after each instruction. It is a small, executable model of your reasoning. Predict first, run second, and explain any difference. Copying code until it works trains recognition rather than independent reasoning. For an unfamiliar program, choose one tiny input and work through it before testing a large case.

WORKED EXAMPLE

Let x = 4; y = x + 2; x = 10. The final values are x = 10 and y = 6. The earlier calculation does not automatically rerun when x changes.

PRACTICE + PASS CRITERION

Create three assignment sequences with at least four statements each. Predict their outputs, then check with Python. Include one swap attempt that fails because it overwrites a needed value. Pass: fix the swap and explain the lost information.

ASSIGNED STUDY

R03: Functions, Variables; R04: An Informal Introduction to Python.

CHECK YOUR UNDERSTANDING

1. After a = 3; b = a; a = 8, what is b for these integers?

2. Why does x = y; y = x fail to swap two different values?

3. What makes prediction useful when a run disagrees?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 01.4 • MODULE 01

Units, budgets and engineering notebooks

Learning goal Use units and simple arithmetic to check game calculations.

A number without a unit can be misleading. Position may be measured in meters or pixels, time in seconds or milliseconds, and speed in units per second. Multiplying speed by elapsed time yields displacement. Adding a time directly to a position has no coherent physical meaning. Naming variables with units makes mistakes visible.

A performance budget is also arithmetic. At a target of f frames per second, the time available per frame is 1000/f milliseconds. This is a planning limit, not a guarantee that the machine will meet it. Record hypotheses, observations and conclusions separately. A measurement describes a particular workload on a particular system, not every possible game.

WORKED EXAMPLE

A speed of 6 units/second for 0.25 seconds gives 1.5 units of travel. At 60 frames/second, the frame interval is about 16.67 milliseconds. A 20 ms frame exceeds that interval.

PRACTICE + PASS CRITERION

Calculate travel at 2, 6 and 10 units/second for 0.1 and 0.5 seconds. Calculate frame intervals for 30, 60 and 120 frames/second. Pass: show units at every step and identify which assumptions are chosen targets.

ASSIGNED STUDY

R03: Variables and arithmetic; R15: Performance Measures.

CHECK YOUR UNDERSTANDING

1. How far does 8 units/second travel in 0.125 seconds?

2. What is the interval for 120 frames/second?

3. Why record hardware and workload with a timing result?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 02

Values, decisions and repetition

Prerequisites Module 01.

Four lessons

02.1 Types and expressions

Choose integers, real-valued numbers, strings and booleans appropriately.

02.2 Conditions and boundary cases

Write decisions with complete and non-overlapping rules.

02.3 Loops and invariants

Use repetition that terminates and preserves a stated property.

02.4 Functions and contracts

Separate a calculation from interaction and state its valid inputs.

Module project

Build a terminal-based resource-gathering simulation with bounded health, inventory and a fixed number of turns. Include invalid-input handling and a complete trace for a five-turn run.

Core references

R03 Harvard University — CS50 Introduction to Programming with Python

R04 Python Software Foundation — Python tutorial and language documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 02.1 • MODULE 02

Types and expressions

Learning goal Choose integers, real-valued numbers, strings and booleans appropriately.

A type determines what operations mean and which values are representable. Integers suit counts; floating-point numbers approximate many real-valued measurements; strings store text; booleans represent true or false. Python performs some conversions automatically, but a text value such as "12" is not the same thing as the integer 12. Input from a text prompt begins as text.

Floating-point arithmetic is finite precision. Some simple decimal fractions cannot be represented exactly in a binary format. Do not begin by adding arbitrary rounding everywhere: first decide whether a quantity is a count, a continuous measurement or an exact monetary amount. Record assumptions about ranges and units before choosing the representation.

WORKED EXAMPLE

"3" + "4" produces the string "34" in Python, while 3 + 4 produces 7. An inventory count of 7.5 swords indicates a modeling problem unless partial items are deliberately supported.

PRACTICE + PASS CRITERION

Write a resource-cost calculator that converts two text inputs to integers and reports total cost. Test zero, a positive count and malformed text. Pass: invalid input produces a useful message rather than an unexplained crash.

ASSIGNED STUDY

R03: Functions, Variables and Exceptions; R04: Numbers and Text.

CHECK YOUR UNDERSTANDING

1. Why are "12" and 12 different?

2. What does "ab" + "cd" produce?

3. Would an exact item count normally use floating point? Why?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 02.2 • MODULE 02

Conditions and boundary cases

Learning goal Write decisions with complete and non-overlapping rules.

A condition selects behavior according to a boolean expression. Comparison operators such as < and >= make boundaries explicit. An if/elif/else chain evaluates conditions in order and takes the first matching branch. Independent if statements may execute more than one branch. These are different designs.

Boundary cases reveal missing rules. For a door that opens at level 5, level 4, level 5 and level 6 are more informative initial tests than three arbitrary large values. Compound conditions use and, or and not. Write parentheses when they improve clarity. Name complex conditions so that the code expresses the game rule rather than making readers reconstruct it.

WORKED EXAMPLE

A purchase is allowed when gold >= price and stock > 0. Having enough gold is insufficient when stock is zero. A zero price still requires a policy for availability.

PRACTICE + PASS CRITERION

Implement a door requiring either a key or both level >= 5 and stamina >= 10. Enumerate all key/no-key cases around the two numeric boundaries. Pass: explain why each case succeeds or fails.

ASSIGNED STUDY

R03: Conditionals; R04: if Statements.

CHECK YOUR UNDERSTANDING

1. Does gold > price allow buying an item when gold equals price?

2. Evaluate False or (True and False).

3. Why test immediately below, at and above a threshold?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 02.3 • MODULE 02

Loops and invariants

Learning goal Use repetition that terminates and preserves a stated property.

A loop repeats a body. A for loop commonly processes a known sequence; a while loop repeats while a condition holds. Every loop needs a reason it will terminate or an explicit design for ongoing operation. A changing progress measure, such as remaining turns, is often the simplest termination argument.

An invariant is a statement that remains true at a chosen point in every iteration. For example, gold never becomes negative if every purchase first checks affordability. Distinguish the number of iterations from the final value of a loop variable. Off-by-one errors often arise from forgetting whether an endpoint is included.

WORKED EXAMPLE

In Python, range(5) supplies 0, 1, 2, 3, 4: five values, not six. To process turn numbers 1 through 5, range(1, 6) makes the intended numbering explicit.

PRACTICE + PASS CRITERION

Simulate ten turns, collecting three units each turn and spending five only when affordable. Print a trace. Pass: exactly ten updates occur and the resource count is never negative. Add a test for zero turns.

ASSIGNED STUDY

R03: Loops; R04: for Statements and range.

CHECK YOUR UNDERSTANDING

1. How many values does range(2, 7) produce?

2. What must change in while remaining > 0 to make progress?

3. Give an invariant for an inventory limited to ten items.

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 02.4 • MODULE 02

Functions and contracts

Learning goal Separate a calculation from interaction and state its valid inputs.

A function packages an operation behind a name and parameters. Returning a value is different from printing it: the caller can use a returned value in another calculation or test. A contract describes valid inputs, promised outputs and relevant side effects. A pure function depends only on its inputs and does not change outside state. Such functions are usually easier to test.

Break a program into responsibilities rather than arbitrary lengths. A movement calculation should not also prompt for a username, write a save file and change global audio settings. Handling invalid inputs is part of the interface. Decide whether to reject, clamp or represent failure; do not silently choose inconsistent policies.

WORKED EXAMPLE

advance(position, velocity, dt) returns position + velocity * dt. With inputs 2, 8 and 0.25, it returns 4. A negative dt should be deliberately supported or rejected, not accidentally ignored.

PRACTICE + PASS CRITERION

Implement advance with a documented nonnegative finite dt contract. Test stationary motion, negative velocity, zero dt and invalid dt. Pass: the function returns a value and the caller controls printing.

ASSIGNED STUDY

R03: Functions and Unit Tests; R04: Defining Functions.

CHECK YOUR UNDERSTANDING

1. What is the result of advance(2, 8, 0.25)?

2. Why is returning a value more reusable than only printing it?

3. What belongs in a function contract besides parameter names?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 03

Organizing data and small programs

Prerequisites Module 02.

Four lessons

03.1 Sequences, dictionaries and identity

Choose collections according to the operations the program needs.

03.2 Decomposition, modules and interfaces

Divide a small game into parts that can be tested independently.

03.3 Files, serialization and validation

Save structured data without assuming every file is trustworthy.

03.4 Classes, composition and state machines

Represent behavior without creating an unnecessary inheritance hierarchy.

Module project

Build a data-driven turn simulator with functions, a list of entities, a validated configuration file and save/load round-trip tests.

Core references

R03 Harvard University — CS50 Introduction to Programming with Python

R04 Python Software Foundation — Python tutorial and language documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 03.1 • MODULE 03

Sequences, dictionaries and identity

Learning goal Choose collections according to the operations the program needs.

A sequence organizes values by position. A dictionary associates keys with values. Use a list when order and iteration matter; use a dictionary when lookup by a meaningful key is central. The container is part of the model, not merely syntax. An entity name may change, so a stable identifier is often safer than using its displayed name as its permanent identity.

Mutable containers can be shared. Assigning a list to another variable does not automatically create an independent deep copy. Mutating a shared object can therefore surprise callers. Before modifying a collection during iteration, understand whether the iterator remains valid and whether elements will be skipped. Building a new collection is often clearer at this stage.

WORKED EXAMPLE

If a = [1, 2] and b = a, then b.append(3) also changes the list seen through a. By contrast, b = list(a) makes a new outer list, but nested mutable objects can still be shared.

PRACTICE + PASS CRITERION

Store five units with stable IDs, names and health. Look up a unit by ID and create a filtered list of living units. Pass: renaming a unit does not break lookup, and the original collection is unchanged by filtering.

ASSIGNED STUDY

R04: Data Structures; R03: Loops and dictionaries.

CHECK YOUR UNDERSTANDING

1. When is a dictionary preferable to a list?

2. Does b = a copy a Python list?

3. Why can a shallow copy still share changes?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 03.2 • MODULE 03

Decomposition, modules and interfaces

Learning goal Divide a small game into parts that can be tested independently.

A module groups related definitions. An interface is the set of operations other code is expected to use. Good boundaries hide unnecessary details while exposing the information a caller needs. Start with a small separation: simulation logic, input handling and presentation. This is enough to prevent many tangles without inventing a large framework.

Dependencies should be intentional. Simulation can accept commands as data rather than directly reading a keyboard. Then the same simulation can run from recorded input, a test or a future network client. Avoid circular dependencies: if every module imports every other module, a local change can become a system-wide problem.

WORKED EXAMPLE

A simulate_turn(state, command) function can run in a terminal game, in a unit test and in a replay tool. It should not need to know which window or controller produced command.

PRACTICE + PASS CRITERION

Refactor the resource simulation into simulation.py and a separate command-line entry point. Pass: importing simulation.py does not start a game or prompt the user, and a test can call its logic directly.

ASSIGNED STUDY

R04: Modules; R03: Libraries and Unit Tests.

CHECK YOUR UNDERSTANDING

1. What is an interface?

2. Why avoid starting a game as a side effect of importing simulation.py?

3. How does command-as-data design help replays?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 03.3 • MODULE 03

Files, serialization and validation

Learning goal Save structured data without assuming every file is trustworthy.

Serialization converts an in-memory representation into a storable or transmittable form. Deserialization reconstructs a representation from that form. The file format needs a schema: which fields exist, their types and their valid ranges. A syntactically valid JSON file can still contain impossible health values or missing required fields.

Treat file contents as inputs, even when you normally create the files yourself. Files can be truncated, edited or come from a different program version. Do not use unrestricted evaluation to load a save. A round-trip test checks that saving and loading preserve the intended information, but it must be complemented by malformed-input tests.

WORKED EXAMPLE

A save with {"version": 1, "health": 50} is structurally clearer than an undocumented sequence of numbers. Validation should reject a string health or an unsupported version with an actionable explanation.

PRACTICE + PASS CRITERION

Create JSON save/load functions for a small state. Test a normal round trip, missing field, wrong type, unsupported version and truncated file. Pass: the original working save remains usable after a failed load.

ASSIGNED STUDY

R04: Input and Output, json documentation linked from the standard library.

CHECK YOUR UNDERSTANDING

1. Does valid JSON imply a valid game state?

2. What does a round-trip test check?

3. Why include a format version?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 03.4 • MODULE 03

Classes, composition and state machines

Learning goal Represent behavior without creating an unnecessary inheritance hierarchy.

A class can combine state and operations that maintain its rules. Encapsulation means that other code uses an interface rather than directly violating those rules. Composition builds a larger object from smaller capabilities. Inheritance describes an is-a relationship and can support substitution, but it is not a requirement for every game object.

A finite state machine models a system using named states and transitions. A character might be idle, moving or stunned. Writing allowed transitions explicitly prevents contradictory combinations such as being both stunned and actively attacking, unless the design intentionally allows that. The model should follow actual behavior, not a fashionable pattern.

WORKED EXAMPLE

A door has Closed and Open states. An Open command succeeds only when its lock policy permits the Closed-to-Open transition. Repeated Open commands should have a documented harmless or rejected result.

PRACTICE + PASS CRITERION

Implement a three-state character controller with a transition table. Pass: test every allowed transition and at least three forbidden transitions. Explain why composition would help add inventory independently of movement.

ASSIGNED STUDY

R04: Classes; R03: Object-Oriented Programming.

CHECK YOUR UNDERSTANDING

1. What does encapsulation protect?

2. Why can independent booleans create contradictory states?

3. When is composition useful?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 04

Debugging, testing and collaboration

Prerequisites Modules 01-03.

Four lessons

04.1 Debugging as an experiment

Find a defect by narrowing evidence rather than making random edits.

04.2 Unit, integration and property tests

Choose tests that challenge the contract instead of repeating the implementation.

04.3 Git, branches and recoverable changes

Use small commits to preserve reasoning and collaborate safely.

04.4 Review, documentation and responsible AI help

Explain a change and verify assistance without surrendering understanding.

Module project

Publish a local tagged release of the terminal simulation with tests, a reproducible bug report, a reviewed change and an evidence-based development journal. A remote public upload is not required.

Core references

R03 Harvard University — CS50 Introduction to Programming with Python

R05 Scott Chacon and Ben Straub — Pro Git

R11 C++ Core Guidelines editors — C++ Core Guidelines

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 04.1 • MODULE 04

Debugging as an experiment

Learning goal Find a defect by narrowing evidence rather than making random edits.

A bug report distinguishes expected behavior from observed behavior and supplies reproducible steps. A debugger lets you pause, inspect variables and follow control flow. Logging records selected events over time. Neither tool replaces a hypothesis: identify what you believe is wrong, choose an observation that could disprove it, and change one relevant thing.

Reduce failing examples. A ten-line reproduction is easier to reason about than a full game. Check the earliest point at which state becomes incorrect, not only where a crash finally happens. Keep the original failure and its fix in a regression test so that future changes do not silently reintroduce it.

WORKED EXAMPLE

If a unit loses health twice, log command receipt and damage application separately. Two damage events could come from duplicate commands or from one command processed twice; the two hypotheses require different fixes.

PRACTICE + PASS CRITERION

Introduce a double-damage bug into a copy of the simulator. Write expected/actual behavior, reduce the reproduction and use a breakpoint or trace to identify the first incorrect update. Pass: fix it with a regression test.

ASSIGNED STUDY

R03: Exceptions and Unit Tests; R11: assertions and error-handling guidance.

CHECK YOUR UNDERSTANDING

1. What information makes a bug report actionable?

2. Why inspect the earliest wrong state?

3. What is a regression test?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 04.2 • MODULE 04

Unit, integration and property tests

Learning goal Choose tests that challenge the contract instead of repeating the implementation.

A unit test checks a small responsibility. An integration test checks that components work together. A property test checks a rule over many inputs, such as inventory never becoming negative after valid purchases. The expected result should come from the specification, a simpler reference implementation or a mathematical property, not a copy of the same potentially wrong algorithm.

Tests are evidence, not proof of all behavior. Passing tests cannot demonstrate untested requirements. Include empty inputs, boundaries, invalid inputs and representative workloads. Keep tests repeatable by controlling randomness and time. A flaky test teaches people to ignore alarms and should be investigated rather than routinely rerun until it passes.

WORKED EXAMPLE

For clamp(x, 0, 100), examples include -1 -> 0, 50 -> 50 and 101 -> 100. A useful property is that every output lies in [0, 100] and clamping an already clamped value changes nothing.

PRACTICE + PASS CRITERION

Write at least twelve tests across motion, resource spending and save/load. Include one property-style loop over many small inputs. Pass: temporarily insert a known bug and demonstrate that a relevant test fails.

ASSIGNED STUDY

R03: Unit Tests; R04: unittest standard-library documentation.

CHECK YOUR UNDERSTANDING

1. How does an integration test differ from a unit test?

2. Why is copying the implementation into the test oracle weak?

3. What does a test suite passing establish?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 04.3 • MODULE 04

Git, branches and recoverable changes

Learning goal Use small commits to preserve reasoning and collaborate safely.

Git records snapshots of tracked files. The working tree is what you edit; the staging area selects the content for the next commit; a commit stores a snapshot and history relationship. A branch is a movable reference to a line of development, not a complete independent copy of every file.

Small commits with meaningful messages make review and recovery easier. Inspect differences before committing. Exclude generated files, credentials and large unnecessary binaries. A remote repository is useful for collaboration and additional copies, but it is not permission to publish someone else's assets or secrets. Learn revert on a practice repository before using destructive history operations on shared work.

WORKED EXAMPLE

A commit message such as "Reject negative purchase quantities; add boundary tests" explains an intent. "Updates" forces a reviewer to reconstruct it from every changed line.

PRACTICE + PASS CRITERION

Initialize the course repository, make three focused commits and implement one feature on a branch. Merge it and practice reverting an intentionally bad commit in a disposable copy. Pass: identify exactly what changed and why.

ASSIGNED STUDY

R05: Getting Started, Git Basics and Basic Branching and Merging.

CHECK YOUR UNDERSTANDING

1. What does the staging area control?

2. Why should secrets not be committed even if later deleted?

3. What makes a useful commit message?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 04.4 • MODULE 04

Review, documentation and responsible AI help

Learning goal Explain a change and verify assistance without surrendering understanding.

A review asks whether a change meets its requirements, preserves contracts and is understandable to the next maintainer. Critique behavior and evidence rather than the person. Documentation should explain setup, decisions and limitations; comments should add intent rather than merely restating each statement.

An AI assistant can explain errors, suggest test cases and compare approaches. Its output is untrusted until checked. Do not submit generated code you cannot explain, and do not expose private data or licensed material without permission. For this course, attempt each quiz unaided, label assisted work and complete a small no-assistance transfer task before marking a skill mastered.

WORKED EXAMPLE

Instead of asking "write the whole engine," ask for three possible causes of a reproducible failure, then test them. Preserve your prediction, the suggestion and the evidence that accepted or rejected it.

PRACTICE + PASS CRITERION

Review a ten-line change using a checklist: requirement, correctness, boundary cases, readability and tests. Write a README that a second person can follow. Pass: explain every changed line and recreate a small related feature without assistance.

ASSIGNED STUDY

R05: Distributed Git; R11: philosophy and interface guidance; course assessment policy.

CHECK YOUR UNDERSTANDING

1. What should a review criticize?

2. Why is compiling AI-generated code insufficient validation?

3. What demonstrates understanding beyond copying a solution?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 05

A complete first game

Prerequisites Modules 01-04.

Four lessons

05.1 The update loop and command model

Separate simulation ticks from input collection and presentation.

05.2 Collision rules and complete behavior

Specify edge cases before implementing a playable rule.

05.3 Deterministic replay and simple measurement

Reproduce a run and measure the work actually performed.

05.4 Finishing, usability and the first portfolio artifact

Deliver a tiny game another person can run and understand.

Module project

Complete Gate A: a deterministic terminal grid game with movement, obstacles, win/loss states, replay, tests and a clean-start README. Use the detailed project brief in the program handbook.

Core references

R20 Carnegie Mellon University — 15-466/666 Computer Game Programming

R22 SDL project — SDL3 documentation

R27 Glenn Fiedler — Fix Your Timestep!

R29 Microsoft — Xbox Accessibility Guidelines

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 05.1 • MODULE 05

The update loop and command model

Learning goal Separate simulation ticks from input collection and presentation.

A simulation tick is one application of the game's update rules. A frame is one produced image. They need not occur at the same rate. Start with a turn-based loop so each command advances exactly one tick. Store commands as values and let update(state, command) apply the rules.

Do not put important state transitions in rendering or sound playback. This separation allows headless testing, where no window is needed. It also makes failures reproducible: the same initial state and command sequence should produce the same state when the simulation is deterministic. Later lessons will address numerical and scheduling obstacles to cross-platform determinism.

WORKED EXAMPLE

For a grid game, command Right can move from (2, 3) to (3, 3) only when the destination is valid. Render reads the resulting grid but performs no additional move.

PRACTICE + PASS CRITERION

Build a 10-by-10 terminal grid with one player and an exit. Pass: one command causes one update, drawing twice changes nothing, and the update function works without printing.

ASSIGNED STUDY

R20: runtime systems course overview; R27: fixed-step concepts, read without copying its implementation.

CHECK YOUR UNDERSTANDING

1. Are a simulation tick and a rendered frame necessarily identical?

2. What should happen when render is called twice without an update?

3. Why is headless simulation useful?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 05.2 • MODULE 05

Collision rules and complete behavior

Learning goal Specify edge cases before implementing a playable rule.

A rule is incomplete when it defines only the happy path. Movement must say what happens at a wall, at the grid boundary, when two commands arrive or after the game is over. A collision system answers questions about overlap or blocked motion, but the game design decides what the answer means.

For this project, use integer grid coordinates and a set of blocked cells. Reject invalid moves without partially changing state. This is a simple transaction: validate, then apply. Separate win/loss detection from input text parsing so a typo does not accidentally consume a game turn unless that is the documented rule.

WORKED EXAMPLE

On a grid with x in 0 through 9, moving right from x = 9 must not yield x = 10. A chosen policy is to keep the old state and report blocked; wrapping is a different valid game rule.

PRACTICE + PASS CRITERION

Add five obstacles, a goal and a terminal win state. Test every edge of the board, blocked movement, reaching the goal and commands after victory. Pass: no partial movement occurs on rejected commands.

ASSIGNED STUDY

R20: game-programming projects; course transition and testing lessons.

CHECK YOUR UNDERSTANDING

1. What is missing from "move right increases x"?

2. Under the reject policy, what happens at x = 9 on a 0-9 grid?

3. Why validate before applying a multi-part state change?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 05.3 • MODULE 05

Deterministic replay and simple measurement

Learning goal Reproduce a run and measure the work actually performed.

A replay stores initial conditions and commands in their intended order. If randomness is used, record the seed and the generator policy as well. Determinism means the relevant outputs match for the same defined inputs and environment. It does not follow merely from setting one random seed when other sources of variability remain.

Measure update cost separately from printing; terminal output can dominate a small program. Use a monotonic timing source for elapsed durations. Repeat a representative workload and report the workload size, repetitions and environment. Avoid claiming that a small terminal benchmark predicts the performance of a future three-dimensional game.

WORKED EXAMPLE

A replay of Right, Right, Down from (0, 0) ends at (2, 1) on an unobstructed grid where Down increases y. A blocked cell at (1, 0) changes that result under the rejection policy.

PRACTICE + PASS CRITERION

Save and replay a twenty-command session. Run the same replay repeatedly and compare final states. Time 10,000 headless updates separately from display. Pass: report exact inputs and no invented performance target.

ASSIGNED STUDY

R27: reproducibility motivation; R04: time and random standard-library references.

CHECK YOUR UNDERSTANDING

1. What information does a replay need besides button names?

2. Why exclude printing from a simulation microbenchmark?

3. Does one matching replay prove determinism for all inputs?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 05.4 • MODULE 05

Finishing, usability and the first portfolio artifact

Learning goal Deliver a tiny game another person can run and understand.

A finished small project teaches integration that disconnected exercises cannot. Define a limited scope and complete it: startup, instructions, gameplay, termination and error messages all matter. Include a way to quit and restart. Avoid relying on color alone to convey important information. Clear controls and readable output are engineering responsibilities even in a terminal game.

A portfolio artifact should include source, tests, run instructions, a short demonstration and a discussion of decisions. Explain what you built yourself and what came from references. Do not inflate a simple project into a claim of production-engine expertise. An honest account of a difficult bug and a measured improvement is more informative than an unsupported adjective.

WORKED EXAMPLE

A useful release note says "Added obstacle rejection and a replay test for the top-right boundary." A useful limitation says "Keyboard commands are turn-based; there is no real-time renderer yet."

PRACTICE + PASS CRITERION

Complete Gate A and ask a tester to start from the README without verbal help. Record three observations and fix at least one usability issue. Pass: the tester can start, play, quit and rerun the tests.

ASSIGNED STUDY

R29: Input, UI navigation and error-message guidelines; R05: tagging releases.

CHECK YOUR UNDERSTANDING

1. What belongs in a clean-start README?

2. Why should color not be the only important cue?

3. What evidence should accompany a claimed improvement?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.